

HWSW Project: pyflate Benchmark Report

1. Overview

Benchmark: pyflate

Description: pyflate is a pure-Python decompression benchmark that supports DEFLATE and bzip2 data. In the pyperformance benchmark, it decompresses the interpreter.tar.bz2 archive. During decompression, the input is processed bit by bit, Huffman codes are decoded into symbols, a Move-To-Front list is updated, and the Burrows-Wheeler Transform is reversed.

Libraries used: pure Python standard library, timing/statistics handled by pyperf.

Data structures: The compressed input is read through bit-buffer classes that provide bit-level access. Huffman codes are stored in a table of code entries, and the original decoder searches this table linearly when decoding each symbol. The Move-To-Front stage maintains a Python list that is reordered whenever a symbol is processed. The BWT reversal stage was not changed in our optimizations.

2. Initial Analysis

Profiling setup: `perf record -F 999 -g -- python3-dbg -m pyperformance run --bench pyflate`, followed by `perf report --stdio`, using a debug build of Python (`python3-dbg`).

Findings: `perf` confirmed that the benchmark spends most of its time executing Python code, but it did not clearly identify the Python-level hotspot.

We therefore profiled the decompression function directly with `cProfile`. This showed that:

`find_next_symbol()` accounted for about **49.6%** of total runtime.
`move_to_front()` accounted for about **9.9%**.

From the source code, we identified two clear optimization targets:

- the linear scan used for Huffman symbol lookup.
- the list slicing and concatenation used in *`move_to_front()`*.

These results directly motivated the two optimizations below.

3. Optimizations

Optimization 1: O(1) Huffman symbol lookup

Idea: `find_next_symbol()` scanned the Huffman table linearly for every decoded symbol.

Change: We precomputed dictionaries that map `(bit_length, code_value)` directly to the matching symbol. During decoding, the function checks only the possible code lengths instead of scanning the full table.

Result: Runtime improved from **1.12 s to 1.07 s**, about **5% faster**.

Conclusion: The lookup optimization helped, but not enough to reach the required 7% improvement. This indicated that another hotspot still needed to be addressed.

Optimization 2: + move_to_front() using pop/insert

Idea: The original implementation rebuilt the list using slicing and concatenation on every call.

Change: We replaced the slicing operation with: `l.insert(0, l.pop(c))`

This performs the same move-to-front operation without creating several temporary lists.

Result: With both optimizations enabled, runtime decreased to **0.887 s**, giving about a **20.8% improvement (1.26× speedup)** over the baseline.

Conclusion: The two optimizations together exceeded the required 7% improvement target.

4. Performance Comparison

4.1 pyperf wall-clock comparison (process-isolated measurements)

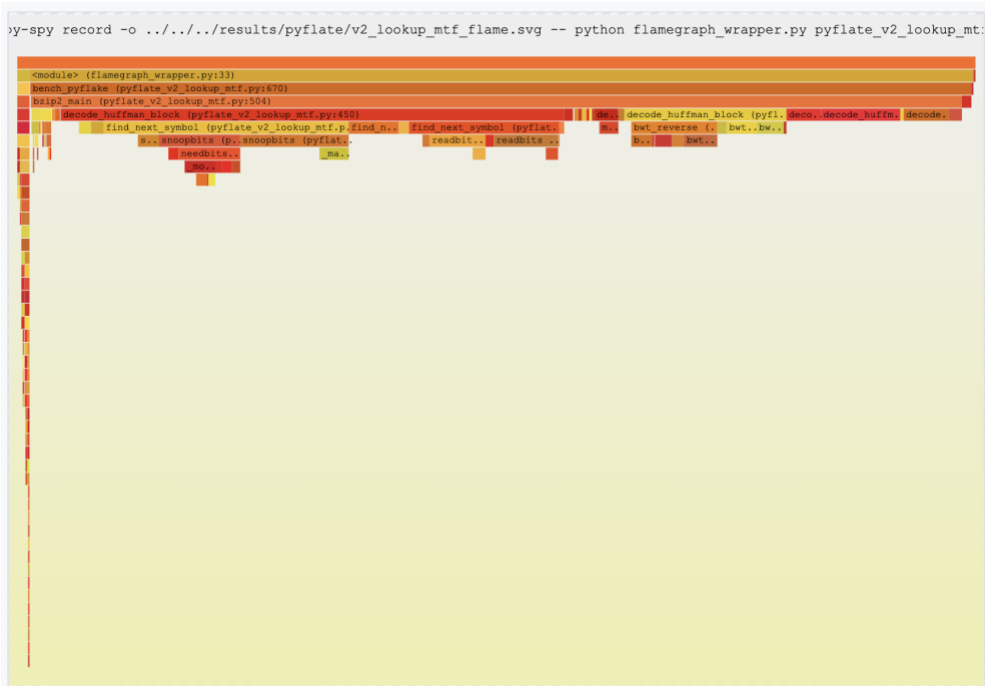
Optimization	Result	Verdict
Baseline (original)	1.12 sec ± 0.01 sec	-
1. Huffman lookup only	1.07 sec ± 0.01 sec	1.05x faster (~5%)
2. + move_to_front	0.887 sec ± 0.006 sec	1.26x faster (~20.8%)

4.2 Flamegraph Insights

Figure 1. Flame graph of the original pyflate benchmark



Figure 2. Flame graph of the optimized (lookup + move_to_front) pyflate benchmark



5. Hardware Acceleration Proposal

5.1 Target

The Huffman symbol-decode loop and the move-to-front update are targeted for hardware acceleration - the same two operations identified as bottlenecks in software, and the same two operations optimized in software. For every symbol decoded (148,271 times in the profiled run), the software performs a table lookup keyed by a variable-length bit pattern and, for most symbols, a move-to-front list update. Both are simple, repetitive, branch-light operations well suited to dedicated combinational/sequential hardware.

5.2 Proposed architecture

The accelerator is implemented in SystemVerilog using:

pyflate_controller.sv — a 9-state FSM that controls bit input, Huffman lookup, MTF update, and output.

pyflate_datapath.sv — stores the current Huffman code and connects the lookup and MTF units.

huffman_lookup.sv — compares the current code against the loaded Huffman table.

move_to_front.sv — updates the 256-entry MTF list.

pyflate_accelerator.sv — top-level module connecting the controller and datapath.

pyflate_mmio.sv — provides simple control/status/output registers.

The current design processes one symbol at a time. Input bits and Huffman table entries are supplied directly through accelerator ports.

5.3 Inputs / outputs

Inputs: The accelerator receives the compressed stream one bit at a time through *input_bit* and *input_valid*. Huffman table entries are loaded separately using *load_table_entry* together with *table_index*, *table_length*, *table_code*, and *table_symbol*. The control inputs are *start* and *init_mtf*.

Outputs: The accelerator outputs the decoded byte through *decoded_symbol* together with *decoded_valid*. It also provides busy and done status signals.

The MMIO wrapper exposes three registers:

- CONTROL — *start*, *init_mtf*
- STATUS — *busy*, *done*, *decoded_valid*
- OUTPUT — *decoded_symbol*

The Huffman table and compressed input bits are provided through direct ports, not through the MMIO registers.

5.4 Hardware/software interface

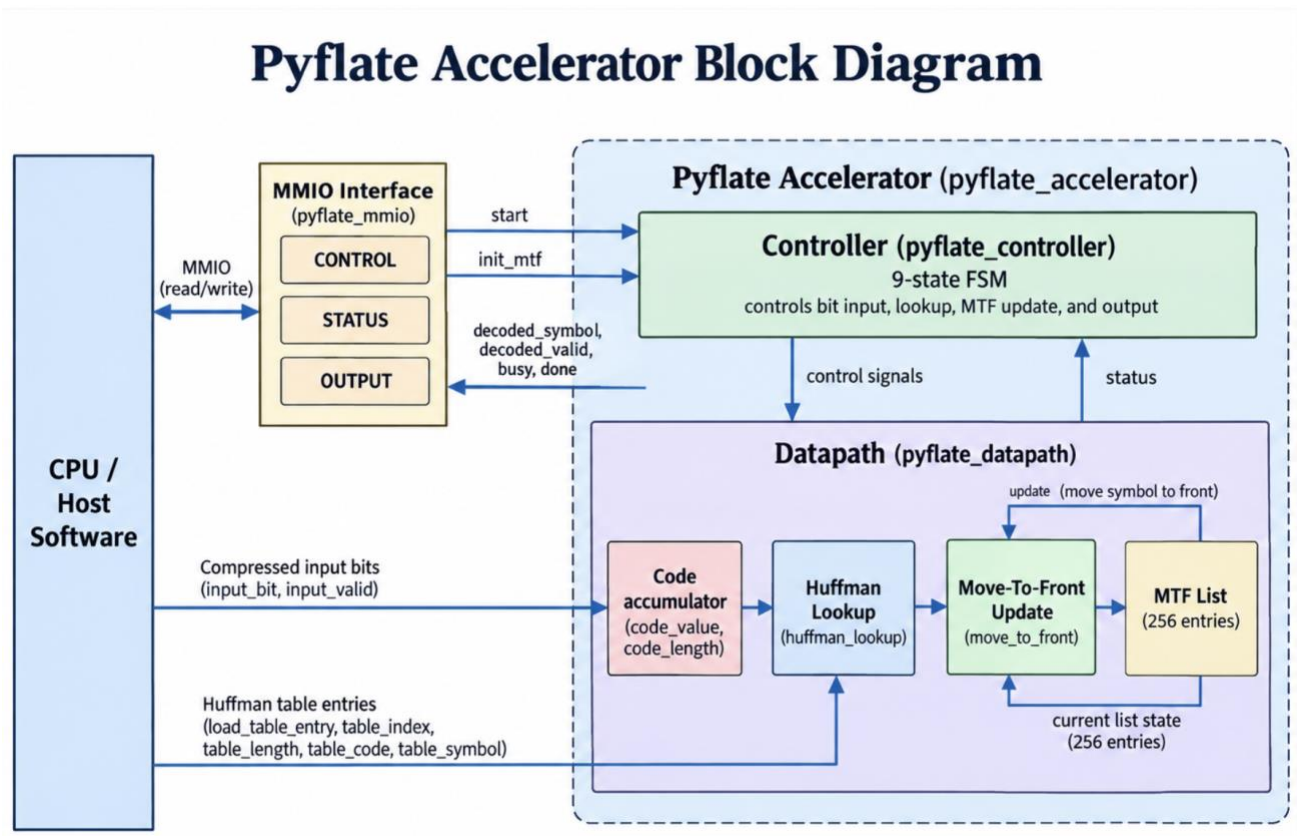
Before decoding starts, the host loads the Huffman table one entry at a time. During decoding, the compressed stream is supplied one bit at a time through the accelerator input ports.

This simple interface matches the current RTL but limits throughput because it requires many host-accelerator handshakes. A future implementation could use buffering or DMA to stream table data and compressed input more efficiently.

5.5 Acceleration justification

Huffman decoding and move-to-front updates are repeated for every decoded symbol, making them suitable for dedicated hardware. The proposed accelerator replaces the software table scan with parallel comparisons and performs the MTF update directly in registers. The design is architectural only; no hardware execution time is reported.

5.6 Block diagram



5.7 Performance / area / power trade-offs

Parallel Huffman lookup.

Comparing many Huffman entries in parallel reduces lookup latency, but requires more comparators and increases area.

Move-to-front update.

Updating the 256-entry list in one cycle improves latency but requires a wide shift/multiplexer network.

Sequential symbol processing.

The current design decodes one symbol at a time. A more pipelined design could improve throughput but would require more control logic.

Host communication.

The current design loads the table entry-by-entry and the input bit-by-bit. This keeps the interface simple but creates significant communication overhead.

6. Conclusion

The `pyflate` benchmark had two main software bottlenecks: Huffman symbol lookup and the move-to-front operation.

Replacing the linear Huffman scan with a precomputed dictionary improved runtime by about **5%**. Optimizing `move_to_front()` as well reduced runtime from **1.12 s to 0.887 s**, giving a total improvement of about **20.8% (1.26×)**.

Based on the same hotspots, we designed a hardware accelerator containing a parallel Huffman lookup unit and a hardware move-to-front unit. The current design processes symbols sequentially and uses a simple direct-port interface for input data and table loading.

The SystemVerilog design is an architectural model and was not synthesized or physically implemented.